

Sequence Type

In Python, sequence is the ordered collection of similar or different data types. Sequences allows to store multiple values in an organized and efficient fashion. There are several sequence types in Python -

- String
- List
- array
- Tuple
- set
- dictionary

Python - String---Immutable data type

A string is a sequence of characters. This conversion of character to a number is called encoding, and the reverse process is decoding. ASCII and Unicode are some of the popular encodings used. In Python, a string is a sequence of Unicode characters. Unicode was introduced to include every character in all languages and bring uniformity in encoding.

How to create a string in Python?

```
In [102]: my_string = 'Hello'
print(my_string)

my_string = "Hello"
print(my_string)

my_string = '''Hello'''
print(my_string)

# triple quotes string can extend multiple lines
my_string = """Hello, welcome to
               the world of Python"""
print(my_string)
```

```
Hello
Hello
Hello
Hello, welcome to
               the world of Python
```

Accessing characters from string

We can access individual characters using indexing and a range of characters using slicing. Index starts from 0. Trying to access a character out of index range will raise an IndexError.

indexing

```
In [20]: str1 = 'python'
print('str = ', str1)
print('str[0] = ', str1[0])
print('str[3] = ', str1[3])
```

```
str = python
str[0] = p
str[3] = h
```

Negative Indexing

The index of -1 refers to the last item, -2 to the second last item and so on.

```
In [21]: #last character
print('str[-1] = ', str1[-1])
#Third last element

print('str[-3] = ', str1[-3])
```

```
str[-1] = n
str[-3] = h
```

Slicing

We can access a range of items in a string by using the slicing operator :(colon).

```
In [23]: #slicing 2nd to 5th character
print('str[1:5] = ', str1[0:3])
#slicing 2nd to 2nd last character
print('str[1:-2] = ', str1[1:-1])
```

```
str[1:5] = pyt
str[1:-2] = ytho
```

How to change or delete a string?

Strings are immutable. This means that elements of a string cannot be changed once they have been assigned. We can simply reassign different strings to the same name.

```
In [106]: str1 = 'python'
          str1[0] = 'P'
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-106-01db4d5e29e7> in <module>
      1 str1 = 'python'
----> 2 str1[0] = 'P'

TypeError: 'str' object does not support item assignment
```

```
In [ ]: str1 = 'python'
        str1="Python"
        print(str1)
```

Concatenation of Two or More Strings

```
In [107]: str1 = 'Hello'
          str2 = 'World!'

          # using +
          print('str1 + str2 = ', str1 + str2)

          # using *
          print('str1 * 3 =', str1 * 3)#The * operator can be used to repeat the string for a given number of times.

          str1 + str2 = HelloWorld!
          str1 * 3 = HelloHelloHello
```

Iterating Through a string

We can iterate through a string using a for loop. Here is an example to count the number of 'l's in a string.

```
In [108]: count = 0
          for letter in 'Hello World':
              if(letter == 'l'):
                  count += 1
          print(count,'letters found')
```

3 letters found

String Membership Test

```
In [109]: 'a' in 'program'
```

Out[109]: True

Python String Formatting

Escape Sequence

```
In [110]: print("He said, "What's there?"")
```

```
File "<ipython-input-110-5b2db8c64782>", line 1
    print("He said, "What's there?"")
                        ^
```

SyntaxError: invalid syntax

One way to get around this problem is to use triple quotes. Alternatively, we can use escape sequences.

```
In [111]: # using triple quotes
print('''He said, "What's there?''')

# escaping single quotes
print('He said, "What\'s there?')

# escaping double quotes
print("He said, \"What's there?\")

# escaping \
print("C:\\Python32\\Lib")
#escaping \n
print("This is printed\nin two lines")
```

```
He said, "What's there?"
He said, "What's there?"
He said, "What's there?"
C:\Python32\Lib
This is printed
in two lines
```

Raw String to ignore escape sequence

Sometimes we may wish to ignore the escape sequences inside a string. To do this we can place `r` or `R` in front of the string.

```
In [112]: print("This is \x48\x45\x58 \ngood example")
print(r"This is \x48\x45\x58 \ngood example")
```

```
This is HEX
good example
This is \x48\x45\x58 \ngood example
```

The format() Method for Formatting Strings

```
In [1]: number = int(input("Enter number "))
name = input("Enter name ")
print("number={0},name={1}".format(number,name))
print("name={1},numner={0}".format(number,name))
print("number=%4d,name=%s"%(number,name))# old style formatting
```

```
Enter number 123
Enter name aaa
number=123,name=aaa
name=aaa,numner=123
number= 123,name=aaa
```

Built-in functions in Python

```
In [40]: str1 = 'python'
print('len(str) = ', len(str1))
```

```
len(str) = 6
```

```
In [37]: x="PYTHON".lower()
print(x)
y='python'.upper()
print(y)
str1="this is demonstration of string built-in functions".split()
print(str1)
str1="this,is,demonstration,of,string,built-in,functions".split(',')
print(str1)
print(' '.join(str1))
print(';'.join(str1))
```

```
python
PYTHON
['this', 'is', 'demonstration', 'of', 'string', 'built-in', 'functions']
['this', 'is', 'demonstration', 'of', 'string', 'built-in', 'functions']
this is demonstration of string built-in functions
this;is;demonstration;of;string;built-in;functions
```

```
In [116]: print('Happy New Year'.find('ew'))  
          'Happy New Year'.replace('Happy', 'Very happy')
```

7

```
Out[116]: 'Very happy New Year'
```

```
In [1]: str1="this is the demonstration of the string built-in functions"  
        print(str1.find("the")) # returns the index of first occurrence of the substring (if found). If not found, it returns -1.  
        print(str1.endswith("functio"))  
        print(str1.startswith("this"))
```

8

False

True

Reversing a string

```
In [2]: def reverse_slicing(s):# way 1  
        return s[::-1]  
        reverse_slicing("python")
```

```
Out[2]: 'n'
```

```
In [43]: s1 = '' #way 2  
        for c in "python":  
            s1 = c + s1  
            print(s1)
```

p

yp

typ

htyp

ohtyp

nohtyp

try it yourself

Python program to check whether string is palindrom or not
python program to find duplicates in string

Write a program which takes a sentence from user and calculates number of digits, letters, uppercase letters, lowercase letter and spaces in sentence.

```
In [120]: str1="Python 1 program to check whether string is palindrom or not"
lower=0
upper=0
digit=0
space=0
for i in str1:
    if(i.isdigit()):
        digit=digit+1
    elif(i.islower()):
        lower=lower+1
    elif(i.isupper()):
        upper=upper+1
    else:
        space=space+1
print(lower)
print(upper)
print(digit)
print(space)
```

```
48
1
1
10
```

list and tuple in python

List is a collection which is ordered and changeable. Allows duplicate members.

Tuple is a collection which is ordered and unchangeable. Allows duplicate members.

Set is a collection which is unordered and unindexed. No duplicate members.

Dictionary is a collection which is unordered, changeable and indexed. No duplicate members.

The list is the most versatile datatype available in Python, which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that the items in a list need not be of the same type.

```
In [44]: # empty list
my_list = []
print(my_list)
# list of integers
my_list = [1, 2, 3]
print(my_list)
# list with mixed data types
my_list = [1, "Hello", 3.4]
print(my_list)
print(type(my_list))
```

```
[]
[1, 2, 3]
[1, 'Hello', 3.4]
<class 'list'>
```

```
In [122]: # nested list
my_list = ["mouse", [8, 4, 6], ['a']]
print(my_list)
```

```
['mouse', [8, 4, 6], ['a']]
```

access elements from a list

- List Index
- Negative Indexing
- slicing and striding

```
In [123]: l1=[1,2,3.3,4,5,6,7]
print(l1[0])
print(l1[3])
print(l1[4])
l2=["Happy", [2, 0, 1, 5],3]
print(l2[0][2]) # accesing element of inner list
```

1
4
5
p

```
In [124]: #negative indexing
print(l1[-1])
print(l1[-3])
```

7
5

slicing and striding

The syntax for range slicing is the following:

[Start index (included): Stop index (excluded):stride]

Slicing can also accept a third parameter, the stride, which refers to how many position you want to move forward after the first position is retrieved from the list.

The value of -1 for the stride allows you to start from the end position.

```
In [125]: #slicing and striding  l1=[1,2,3.3,4,5,6,7]
print(l1[0:3]) #accessing first three element
print(l1[:-1]) # accessing all elements except last

print(l1[2:]) # accessing all elements starting with 3rd element
print(l1[:]) # accessing all elements
print(l1[0:5:2]) #accessing elements from first to fifth with step 2
print(l1[::-1]) # reads in reverse
```

```
[1, 2, 3.3]
[1, 2, 3.3, 4, 5, 6]
[3.3, 4, 5, 6, 7]
[1, 2, 3.3, 4, 5, 6, 7]
[1, 3.3, 5]
[7, 6, 5, 4, 3.3, 2, 1]
```

```
In [126]: #list is mutable
l1[2]=3
print(l1)
l1.append(8)
print(l1)
l1.extend([9,10])
print(l1)
```

```
[1, 2, 3, 4, 5, 6, 7]
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
In [127]: # Concatenating and repeating lists
odd = [1, 3, 5]

print(odd + [9, 7, 5])

print(["re"] * 3)
```

```
[1, 3, 5, 9, 7, 5]
['re', 're', 're']
```

```
In [128]: odd.insert(2,4) # inserts at desired position
print(odd)
```

```
[1, 3, 4, 5]
```

In [136]: *# Deleting list items based on index*

```
l2=[1,2,3,4,5,[6,7,8]]
del(l2[0])
print(l2)
del(l2[0:2])
print(l2)
del(l2[2][0])
print(l2)
```

```
[2, 3, 4, 5, [6, 7, 8]]
[4, 5, [6, 7, 8]]
[4, 5, [7, 8]]
```

In [147]: *#deleting the given item*

```
my_list = ['p','r','o','b','l','e','m']
my_list.remove('p')
print(my_list)
```

```
['r', 'o', 'b', 'l', 'e', 'm']
```

In [148]: *#The pop() method removes and returns the last item if the index is not provided.
#This helps us implement lists as stacks (first in, last out data structure).*

```
my_list.pop()
print(my_list)
my_list.pop(1)
print(my_list)
my_list.clear()
print(my_list)
```

```
['r', 'o', 'b', 'l', 'e']
['r', 'b', 'l', 'e']
[]
```

```
In [150]: # Python List methods
my_list = [3, 8, 1, 6, 0, 8, 4]
print(my_list.count(8))
my_list.sort()
print(my_list)
my_list.reverse()
print(my_list)
```

```
2
[0, 1, 3, 4, 6, 8, 8]
[8, 8, 6, 4, 3, 1, 0]
```

List Comprehension: Elegant way to create new List

The basic syntax uses square brackets.

```
[ expression for item in list if conditional ]
```

This is equivalent to:

```
for item in list:
```

```
    if conditional:
```

```
        expression
```

```
if condition is used to filter out result.
```

```
In [8]: x = [i for i in range(10)]
print(x)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [10]: squares = [x**2 for x in range(10)]
print(squares)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
In [11]: #Multiplying parts of a List
list1 = [3,4,5]
multiplied = [item*3 for item in list1]
print(multiplied)
```

```
[9, 12, 15]
```

```
In [13]: [x.lower() for x in ["A","B","C"]]
```

```
Out[13]: ['a', 'b', 'c']
```

```
In [3]: num_list = [y for y in range(11) if y % 2 == 0 if y % 5 == 0]
print(num_list)
```

```
[0, 10]
```

```
In [17]: #Using list comprehension in functions
def double(x):
    return x*2
d=[double(x) for x in range(10)]
print(d)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

```
In [19]: [x+y for x in [10,30,50] for y in [20,40,60]]
```

```
Out[19]: [30, 50, 70, 50, 70, 90, 70, 90, 110]
```

You can place the conditional at the end of the statement for simple filtering, but what if you want to change a member value instead of filtering it out? In this case, it's useful to place the conditional near the beginning of the expression:

```
new_list = [expression (if conditional) for member in iterable]
```

```
In [42]: l1=[1,2,-3,4,-5,6,7,-8]
l2=[x if x>0 else 0 for x in l1]
print(l2)
```

```
[1, 2, 0, 4, 0, 6, 7, 0]
```

Tuple in Python

In Python, a tuple is similar to List except that the objects in tuple are immutable which means we cannot change the elements of a tuple once assigned. On the other hand, we can change the elements of a list.

```
In [1]: #creating Tuple
t1=()
print(t1)
t1=(1,2,3)
print(t1)
t1=(1,"hello",2.2)
print(t1)
t1=(2)
print(t1)
print(type(t1))
t1=(2,) #tuple with one element
print(t1)
t1=2, #parenthesis are optional
print(t1)

()
(1, 2, 3)
(1, 'hello', 2.2)
2
<class 'int'>
(2,)
(2,)
```

Accessing tuple elements

```
In [12]: #1. Indexing
t1=('p','y','t','h','o','n')
print(t1[0])
print(t1[3])
#2. Negative Indexing
print(t1[-1])
print(t1[-2])
```

p
h
n
o

```
In [14]: # nested tuple
n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))

# nested index
print(n_tuple[0][2])
print(n_tuple[1][1])
```

u
4

```
In [16]: #3. Slicing
print(t1[0:3])
print(t1[:-1])
print(t1[2:])
```

('p', 'y', 't')
('p', 'y', 't', 'h', 'o')
('t', 'h', 'o', 'n')

Changing a Tuple

In [18]: `t1[0]='P'`

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-18-fcd31512151a> in <module>  
----> 1 t1[0]='P'  
  
TypeError: 'tuple' object does not support item assignment
```

In [19]: `del t1[2]`

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-19-cad7b2ca8ce3> in <module>  
----> 1 del t1[2]  
  
TypeError: 'tuple' object doesn't support item deletion
```

In [22]: `t2=(1,2)`
`del t2`
`print(t2)`

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-22-d5c51684cb80> in <module>  
      1 t2=(1,2)  
      2 del t2  
----> 3 print(t2)  
  
NameError: name 't2' is not defined
```

```
In [36]: #concatenating two tuples
print((1, 2, 3) + (4, 5, 6))
#repeat tuple
print((1,2,3)*2)
#tuple membership
print(2 in (1,2,3))
```

(1, 2, 3, 4, 5, 6)

(1, 2, 3, 1, 2, 3)

True

Iterating Through a Tuple

```
In [25]: for name in ('Antra', 'Aarav'):
          print("Hello", name)
```

Hello Antra

Hello Aarav

The tuple() method

```
In [2]: t2=tuple((1,2,3,3))
print(t2)
l1=[1,2,3,4]
t3=tuple(l1) # converts list to tuple
print(t3)
```

(1, 2, 3, 3)

(1, 2, 3, 4)

Tuple methods

```
In [3]: print(t2.count(3))
print(t2.index(3))
print("len=",len(t2))
print("max=",max(t2))
```

```
2
2
len= 4
max= 3
```

Array in Python

- Arrays are popular in most programming languages like Java, C/C++. However, in Python, they are not that common. When people talk about Python arrays, more often than not, they are talking about Python lists.
- An array of numeric values is supported in Python by the array module.
- We can treat lists as arrays. However, we cannot constrain the type of elements stored in a list. but in array it has same type of elements.

```
In [7]: a = [1, 3.5, "Hello"]
print(a)
import array as arr
#a=arr.array('d',[1, 3.5, "Hello"]) # raise an error
```

```
[1, 3.5, 'Hello']
```

Creating Python Arrays

- Here, we created an array of float type. The letter d is a type code. This determines the type of the array during creation.
- i for integer
- f for float
- d for double

```
In [8]: import array as arr  
a = arr.array('d', [1.1, 3.5, 4.5])  
print(a)
```

```
array('d', [1.1, 3.5, 4.5])
```

Accessing Python Array Elements

```
In [9]: import array as arr  
a = arr.array('i', [2, 4, 6, 8])  
  
print("First element:", a[0])  
print("Second element:", a[1])  
print("Last element:", a[-1])
```

```
First element: 2  
Second element: 4  
Last element: 8
```

Slicing Python Arrays

```
In [10]: import array as arr  
  
numbers_list = [2, 5, 62, 5, 42, 52, 48, 5]  
numbers_array = arr.array('i', numbers_list)  
  
print(numbers_array[2:5]) # 3rd to 5th  
print(numbers_array[:5]) # beginning to 4th
```

```
array('i', [62, 5, 42])  
array('i', [2, 5, 62])
```

Changing and Adding Elements

- Arrays are mutable; their elements can be changed in a similar way like lists.

```
In [17]: import array as arr

numbers = arr.array('i', [1, 2, 3, 5, 7, 10])
numbers[0] = 0
print(numbers)
numbers[2:5] = arr.array('i', [4, 6, 8])
print(numbers)
```

```
array('i', [0, 2, 3, 5, 7, 10])
array('i', [0, 2, 4, 6, 8, 10])
```

```
In [18]: numbers.append(11)
numbers.extend([12,13])
print(numbers)
```

```
array('i', [0, 2, 4, 6, 8, 10, 11, 12, 13])
```

Removing Python Array Elements

```
In [21]: import array as arr

number = arr.array('i', [1, 2, 3, 3, 4])
del number[2]
print(number)
number.remove(4)
print(number)
del number ## deleting entire array
#print(number) raise error
```

```
array('i', [1, 2, 3, 4])
array('i', [1, 2, 3])
```